

AI Assisted Coding

Assignment – 13.5

Name: K. Deekshith

RollNo: 2303A51414

Batch: 21

Task Description #1 (Refactoring – Removing Global Variables)

- Task: Use AI to eliminate unnecessary global variables from the code.

- Instructions:

- o Identify global variables used across functions.

- o Refactor the code to pass values using function parameters.

- o Improve modularity and testability.

- Sample Legacy Code:

```
rate = 0.1
```

```
def calculate_interest(amount):
```

```
    return amount * rate
```

```
print(calculate_interest(1000))
```

- Expected Output:

- o Refactored version passing rate as a parameter or using a configuration structure.

Code:

- o Logical simplification

- o Maintainability

Legacy Code:

```
score = 78
```

```
if score >= 90:
```

```
    print("Excellent")
```

```
else:
```

```
    if score >= 75:
```

```
        print("Very Good")
```

```
    else:
```

```
        if score >= 60:
```

```
            print("Good")
```

```
        else:
```

```
            print("Needs Improvement")
```

Expected Outcome:

- o Flattened logic using guard clauses or a mapping-based approach.

Code:

```
tempRun.py > ...
1  '''Task Description #2 : (Refactoring Deeply Nested Conditionals)
2  • Task: Use AI to refactor deeply nested if-elif-else logic into a
3  cleaner structure.
4  • Focus Areas:
5  o Readability
6  o Logical simplification
7  o Maintainability
8  Legacy Code:
9  score = 78
10 if score >= 90:
11     print("Excellent")
12 else:
13     if score >= 75:
14         print("Very Good")
15     else:
16         if score >= 60:
17             print("Good")
18         else:
19             print("Needs Improvement")
20 Expected Outcome:
21 o Flattened logic using guard clauses or a mapping-based
22 approach.'''
23 score = 78
24 if score >= 90:
25     print("Excellent")
26 elif score >= 75:
27     print("Very Good")
28 elif score >= 60:
29     print("Good")
30 else:
31     print("Needs Improvement")
```

Output:

```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_AssistantCoding/tempRun.py
Very Good
PS E:\3_2\AI_AssistantCoding>
```

Observation:

1. The original code had deeply nested if-else statements, which made it less readable and harder to maintain.
2. The refactored code uses elif statements to flatten the logic, improving readability and maintainability.
3. The refactored code is more concise and easier to understand, as it eliminates unnecessary nesting and clearly shows the different conditions being evaluated.

Task 3 (Refactoring Repeated File Handling Code)

- Task: Use AI to refactor repeated file open/read/close logic.

- Focus Areas:

- o DRY principle
- o Context managers
- o Function reuse

Legacy Code:

```
f = open("data1.txt")
```

```
print(f.read())
```

```
f.close()
```

```
f = open("data2.txt")
```

```
print(f.read())
```

```
f.close()
```

Expected Outcome:

- o Reusable function using with open() and parameters.

Code:

```
LabTest01.py  tempRun.py X
tempRun.py > ...
1  '''Task 3 (Refactoring Repeated File Handling Code)
2  • Task: Use AI to refactor repeated file open/read/close logic.
3  • Focus Areas:
4  o DRY principle
5  o Context managers
6  o Function reuse
7  Legacy Code:
8  f = open("data1.txt")
9  print(f.read())
10 f.close()
11 f = open("data2.txt")
12 print(f.read())
13 f.close()
14 Expected Outcome:
15 o Reusable function using with open() and parameters.'''
16 def read_and_print_file(file_name):
17     with open(file_name, 'r') as f:
18         print(f.read())
19 # Refactored code using the reusable function
20 read_and_print_file("data1.txt")
21 read_and_print_file("data2.txt")
22
```

Output:

```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/windowsApps/python3.11.exe e:/3_2/AI_Assist
antCoding/tempRun.py
Hello Hi
Hello Bye
PS E:\3_2\AI_AssistantCoding>
```

Observation:

1. The refactored code eliminates redundancy by encapsulating the file handling logic within a reusable function, adhering to the DRY principle.
2. The use of context managers (with open()) ensures that files are properly closed after their operations, enhancing code safety and readability.
3. The function read_and_print_file() can be easily reused for any file, making the code more modular and maintainable.

Task 4 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.

- Focus Areas:

- o Time complexity

- o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
```

```
name = input("Enter username: ")

found = False

for u in users:

    if u == name:

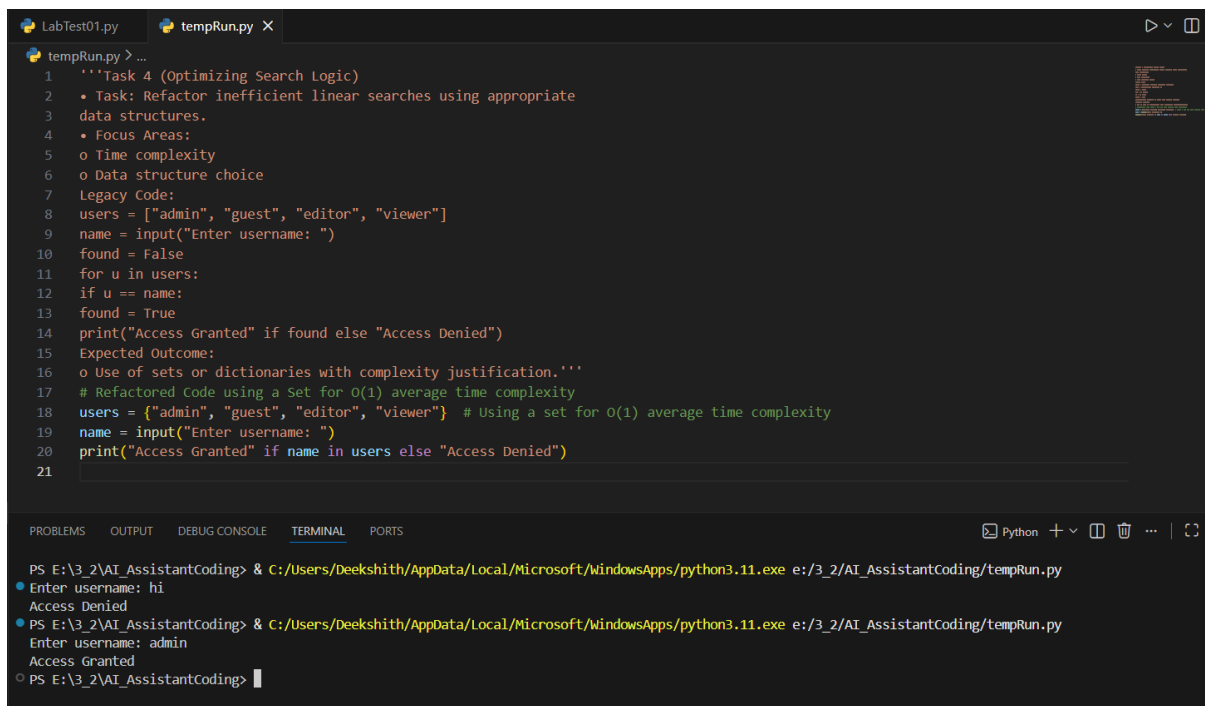
        found = True

print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

- o Use of sets or dictionaries with complexity justification.

Code and Output:



The screenshot displays a code editor with two tabs: 'LabTest01.py' and 'tempRun.py'. The 'tempRun.py' tab is active, showing the following Python code:

```
1 '''Task 4 (Optimizing Search Logic)
2 • Task: Refactor inefficient linear searches using appropriate
3 data structures.
4 • Focus Areas:
5 o Time complexity
6 o Data structure choice
7 Legacy Code:
8 users = ["admin", "guest", "editor", "viewer"]
9 name = input("Enter username: ")
10 found = False
11 for u in users:
12     if u == name:
13         found = True
14 print("Access Granted" if found else "Access Denied")
15 Expected Outcome:
16 o Use of sets or dictionaries with complexity justification.'''
17 # Refactored Code using a Set for O(1) average time complexity
18 users = {"admin", "guest", "editor", "viewer"} # Using a set for O(1) average time complexity
19 name = input("Enter username: ")
20 print("Access Granted" if name in users else "Access Denied")
21
```

Below the code editor, the terminal output is visible, showing the execution of the script with two test cases:

```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_AssistantCoding/tempRun.py
Enter username: hi
Access Denied
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_AssistantCoding/tempRun.py
Enter username: admin
Access Granted
PS E:\3_2\AI_AssistantCoding>
```

Observation:

1. The original code used a list and a linear search, resulting in $O(n)$ time complexity.
2. The refactored code uses a set, which allows for $O(1)$ average time complexity for membership testing.
3. This change significantly improves the efficiency of the search operation, especially as the number of users increases.

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

- Focus Areas:

- o Object-Oriented principles

- o Encapsulation

Legacy Code:

```
salary = 50000
```

```
tax = salary * 0.2
```

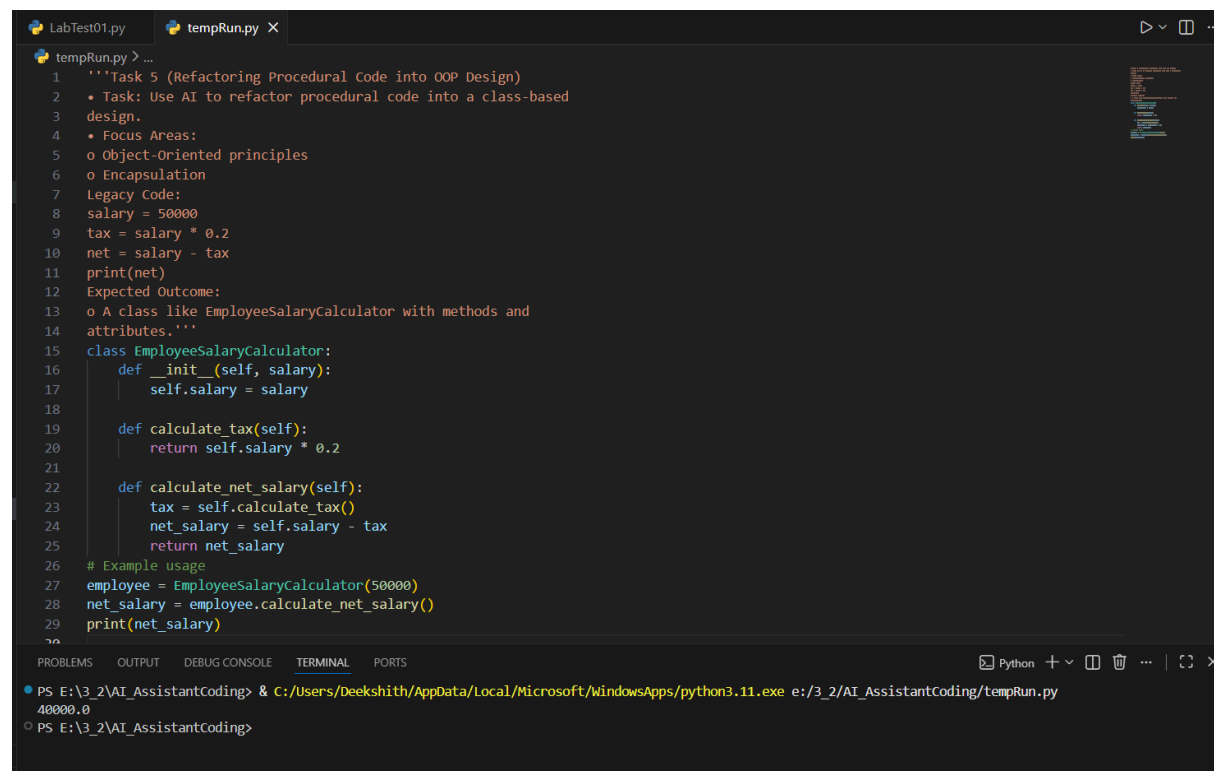
```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.

Code and Output:



The screenshot shows a code editor with two tabs: 'LabTest01.py' and 'tempRun.py'. The 'tempRun.py' tab is active, displaying the following code:

```
1 '''Task 5 (Refactoring Procedural Code into OOP Design)
2 • Task: Use AI to refactor procedural code into a class-based
3 design.
4 • Focus Areas:
5 o Object-Oriented principles
6 o Encapsulation
7 Legacy Code:
8 salary = 50000
9 tax = salary * 0.2
10 net = salary - tax
11 print(net)
12 Expected Outcome:
13 o A class like EmployeeSalaryCalculator with methods and
14 attributes.'''
15 class EmployeeSalaryCalculator:
16     def __init__(self, salary):
17         self.salary = salary
18
19     def calculate_tax(self):
20         return self.salary * 0.2
21
22     def calculate_net_salary(self):
23         tax = self.calculate_tax()
24         net_salary = self.salary - tax
25         return net_salary
26
27 # Example usage
28 employee = EmployeeSalaryCalculator(50000)
29 net_salary = employee.calculate_net_salary()
30 print(net_salary)
```

The bottom of the screenshot shows the terminal output:

```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_AssistantCoding/tempRun.py
40000.0
PS E:\3_2\AI_AssistantCoding>
```

Observation:

1. The code has been refactored into a class-based design, encapsulating the salary calculation logic within the EmployeeSalaryCalculator class.
2. The class has an initializer (__init__) that takes the salary as an argument and stores it as an instance attribute.
3. The class has two methods: calculate_tax() to compute the tax based on the salary, and calculate_net_salary() to compute the net salary by subtracting the tax from the original salary.

Task 6 (Refactoring for Performance Optimization)

- Task: Use AI to refactor a performance-heavy loop handling large data.

- Focus Areas:

- o Algorithmic optimization

- o Use of built-in functions

Legacy Code:

```
total = 0
```

```
for i in range(1, 1000000):
```

```
    if i % 2 == 0:
```

```
        total += i
```

```
print(total)
```

Expected Outcome:

- o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Code and Output:


```
1 '''Task 6 (Refactoring for Performance Optimization)
2 • Task: Use AI to refactor a performance-heavy loop handling
3 large data.
4 • Focus Areas:
5 o Algorithmic optimization
6 o Use of built-in functions
7 Legacy Code:
8 total = 0
9 for i in range(1, 1000000):
10     if i % 2 == 0:
11         total += i
12     print(total)
13 Expected Outcome:
14 o Optimized logic using mathematical formulas or
15 comprehensions, with time comparison.'''
16 # Optimized code using mathematical formula for the sum of even numbers
17 n = 999998 # Largest even number less than 1,000,000
18 # The sum of the first k even numbers is given by the formula: k * (k + 1)
19 k = n // 2 # Number of even numbers up to n
20 total = k * (k + 1) # Using the formula to calculate the sum of even numbers
21 print(total)
22
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python + -

PS E:\3_2\AI_AssistantCoding> C:\Users\Deekshith\AppData\Local\Microsoft\WindowsApps\python3.11.exe e:/3_2/AI_AssistantCoding/tempRun.py
249999500000
PS E:\3_2\AI_AssistantCoding>

Observation:

1. The optimized code calculates the sum of even numbers up to 999,998 using a mathematical formula, which significantly reduces the time complexity from $O(n)$ to $O(1)$.
2. This approach eliminates the need for a loop, making it much faster, especially for large values of n .
3. The original code iterates through all numbers up to 1,000,000, checking for even numbers and summing them, which is inefficient compared to the optimized version.

Task 7 (Removing Hidden Side Effects)

- Task: Refactor code that modifies shared mutable state.

- Focus Areas:

- o Functional-style refactoring

- o Predictability

Legacy Code:

```
data = []
```

```
def add_item(x):
```

```
    data.append(x)
```

```
add_item(10)
```

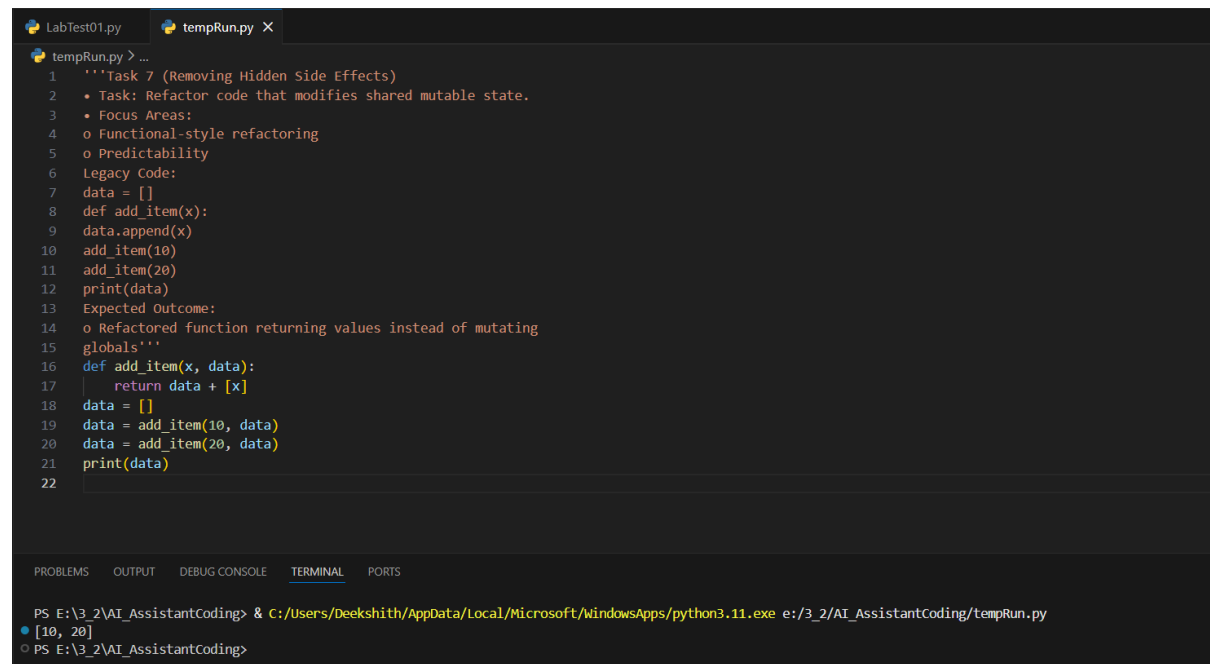
```
add_item(20)
```

```
print(data)
```

Expected Outcome:

- o Refactored function returning values instead of mutating globals

Code and Output:



```
tempRun.py > ...
1 '''Task 7 (Removing Hidden Side Effects)
2 • Task: Refactor code that modifies shared mutable state.
3 • Focus Areas:
4 o Functional-style refactoring
5 o Predictability
6 Legacy Code:
7 data = []
8 def add_item(x):
9     data.append(x)
10    add_item(10)
11    add_item(20)
12    print(data)
13    Expected Outcome:
14    o Refactored function returning values instead of mutating
15    globals'''
16    def add_item(x, data):
17        return data + [x]
18    data = []
19    data = add_item(10, data)
20    data = add_item(20, data)
21    print(data)
22
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_AssistantCoding/tempRun.py
[10, 20]
PS E:\3_2\AI_AssistantCoding>
```

Observation:

1. The `add_item` function now takes an additional parameter 'data' and returns a new list instead of modifying the global variable.
2. The global variable 'data' is updated by assigning the result of the `add_item` function back to it, ensuring that the original list is not mutated.
3. This refactored code is more predictable and easier to test, as it avoids side effects and relies on pure functions that return new values instead of modifying existing ones.

Task 8 (Refactoring Complex Input Validation Logic)

- Task: Use AI to simplify and modularize complex validation rules.
- Focus Areas:
 - o Readability

o Testability

Legacy Code:

```
password = input("Enter password: ")

if len(password) >= 8:

if any(c.isdigit() for c in password):

if any(c.isupper() for c in password):

print("Valid Password")

else:

print("Must contain uppercase")

else:

print("Must contain digit")

else:

print("Password too short")
```

Expected Outcome:

o Separate validation functions with clear responsibility

Code:

```
tempRun.py > -
1  """Task 8 (Refactoring Complex Input Validation Logic)
2  * Task: Use AI to simplify and modularize complex validation
3  rules.
4  * Focus Areas:
5  o Readability
6  o Testability
7  Legacy Code:
8  password = input("Enter password: ")
9  if len(password) >= 8:
10 if any(c.isdigit() for c in password):
11 if any(c.isupper() for c in password):
12 print("Valid Password")
13 else:
14 print("Must contain uppercase")
15 else:
16 print("Must contain digit")
17 else:
18 print("Password too short")
19 Expected Outcome:
20 o Separate validation functions with clear responsibility
21 """
22 def is_valid_length(password):
23     return len(password) >= 8
24
25 def has_digit(password):
26     return any(c.isdigit() for c in password)
27
28 def has_uppercase(password):
29     return any(c.isupper() for c in password)
30
31 def is_valid_password(password):
32     if not is_valid_length(password):
33         print("Password too short")
34         return False
35     if not has_digit(password):
36         print("Must contain digit")
37     return False
```

```
38     if not has_uppercase(password):
39         print("Must contain uppercase")
40         return False
41     print("Valid Password")
42     return True
43 password = input("Enter password: ")
44 is_valid_password(password)]
```

Output:

```
• PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_AssistantCoding/tempRun.py
Enter password: Deekshith
Must contain digit
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_AssistantCoding/tempRun.py
• Enter password: Deekshith7
Valid Password
○ PS E:\3_2\AI_AssistantCoding> █
```

Observation:

1. The code is now modularized with separate functions for each validation rule, improving readability and maintainability.
2. The main validation logic is centralized in the `is_valid_password` function, making it easier to test and modify individual validation rules without affecting the overall structure.
3. The use of descriptive function names enhances the clarity of the code, making it easier for developers to understand the purpose of each function at a glance.